Dylan Francis DAAN 682
05APR2026

Homework_11:

Code:

```
#
========S2==========================================================================
=
# DAAN: 682: Data Analytics Programming in Python
# Author: Dylan Francis
# Title: Homework_11: Unsupervised learning with Skikit-learn: Clustering
#
==============================================================================
# 1.) Perform data exploratory data analysis and interpret the results. (10 points)
# 2.) Apply K-means clustering to group the seed data. Experiment with different values of k and
use unsupervised metrics such as the silhouette score to determine to optimal number of
clusters. Compare the results with the ground truth number of clusters. Visualize the clustering
result. (30 points)
# 3.) Apply Hierarchical clustering using different linkage methods. Compare the results and
determine which linkage method gives the best clustering result. (30 points)
# 4.) Apply DBSCAN clustering to the seed data. Experiment with different values of eps and
min_samples and find the combination of eps and min_samples producing the best clustering
result based on the number of clusters formed and noise points identified. (30 points)
#
#
==============================================================================
import os
#setting this up to avoid the warnings that come up while operating on windows
os.environ["OMP_NUM_THREADS"] = "1"
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
from sklearn.metrics import silhouette_score, adjusted_rand_score
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import numpy as np

path = r"C:\Users\dylan\OneDrive\Documents\GRAD_SCHOOL\DAAN_682\HOMEWORK_11"
os.chdir(path)
#setting this up to avoid the warnings that come up while operating on windows
os.environ["OMP_NUM_THREADS"] = "1"

# Load dataset
seeds_dataset = pd.read_csv("seeds_dataset.csv", header=None)
#assign column headers since the data didn't have any
```

```
columns = ["area", "perimeter", "compactness", "length_kernel",
        "width_kernel", "asymmetry_coefficient", "groove_length", "label"]
seeds_dataset.columns = columns

# breaking up the data
x = seeds_dataset.iloc[:, :-1]
y_true = seeds_dataset["label"]

# 1.) Data Exploration
print(f"\nThere are {seeds_dataset.shape[0]} rows, and {seeds_dataset.shape[1]} columns.")
print("\n======= DATASET INFO ======")
print(x.info())
print("\n======= SUMMARY STATS ======")
print(x.describe())
print("\n======= CORRELATION MATRIX ======")
corr_matrix = x.corr()
print(corr_matrix)
print("\n===== MISSING VALUES =====")
print(x.isnull().sum())
print("\n===== DUPLICATE ROWS =====")
print(f"The total number of duplicate rows is: {x.duplicated().sum()}")

# Histograms
x.hist(figsize=(10, 8), bins=15)
plt.suptitle("Feature Distributions", fontsize=16)
plt.tight_layout()
plt.show()

# Boxplot
plt.figure(figsize=(12, 6))
sns.boxplot(data=x)
plt.title("Boxplot of All Features")
plt.xticks(rotation=45)
plt.show()

# Correlation Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()

# Pairplot
sns.pairplot(x)
plt.suptitle("Pairwise Feature Relationships", y=1.02)
```

```
plt.show()
#
==============================================================================

#2. K means

#Scaling the data / normalize
scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)

# 3. Find Optimal K using Silhouette Score
k_values = range(2, 10)
silhouette_scores = []

for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=13, n_init=10)
    labels = kmeans.fit_predict(x_scaled)
    score = silhouette_score(x_scaled, labels)
    silhouette_scores.append(score)

# Plot silhouette scores
plt.figure(figsize=(8, 5))
plt.plot(k_values, silhouette_scores, marker='x')
plt.title("Silhouette Score vs. Number of Clusters (k)")
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Silhouette Score")
plt.grid()
plt.show()

# determining best k
optimal_k = k_values[np.argmax(silhouette_scores)]
print(f"Optimal k based on silhouette score: {optimal_k}")

kmeans = KMeans(n_clusters=optimal_k, random_state=13, n_init=10)
clusters = kmeans.fit_predict(x_scaled)

# Compare with Truth
ari = adjusted_rand_score(y_true, clusters)
print(f"Adjusted Rand Index (comparison with true labels): {ari:.3f}")

print(f"True number of clusters: {len(np.unique(y_true))}")
print(f"KMeans clusters: {optimal_k}")
if optimal_k != len(np.unique(y_true)):
    print("The k mean model is not yielding expected number of clusters")
```

```
# Visualization using PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(x_scaled)

plt.figure(figsize=(12, 5))

# Plot KMeans clusters
plt.subplot(1, 2, 1)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, cmap='viridis')
plt.title(f"KMeans Clusters (k={optimal_k})")

# Plot ground truth
plt.subplot(1, 2, 2)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y_true, cmap='viridis')
plt.title("Ground Truth Labels")
plt.show()

#
==============================================================================
# 3.) Hierarchical clustering
linkage_methods = ["ward", "average", "complete"]
results = {}

for linkage in linkage_methods:
    model = AgglomerativeClustering(n_clusters=3, metric = 'euclidean', linkage=linkage)
    labels = model.fit_predict(x_scaled)

    sil_score = silhouette_score(x_scaled, labels)
    ari = adjusted_rand_score(y_true, labels)

    results[linkage] = {
        "silhouette": sil_score,
        "ari": ari,
        "labels": labels
    }
print("Linkage Method Comparison:\n")
for method, metrics in results.items():
    print(f"{method.upper()} | Silhouette: {metrics['silhouette']:.3f} | ARI: {metrics['ari']:.3f}")


# Find Best Linkage
best_method_sil = max(results, key=lambda x: results[x]["silhouette"])
best_method_ari = max(results, key=lambda x: results[x]["ari"])
```

```
if best_method_sil == best_method_ari:
    print(f"\nBest linkage method (by silhouette and AIR score) is: {best_method_sil.upper()}")


# Visualization through PCA
pca = PCA(n_components=2)
x_pca = pca.fit_transform(x_scaled)

plt.figure(figsize=(12, 8))

for i, method in enumerate(linkage_methods, 1):
    plt.subplot(2, 2, i)
    plt.scatter(X_pca[:, 0], X_pca[:, 1],
                c=results[method]["labels"],
                cmap="viridis")
    plt.title(f"{method.upper()} Linkage")

plt.tight_layout()
plt.show()
#
==============================================================================
# 3.) DBScan Clustering

db_result = []

epses = np.arange(0.7, 1.1, 0.05)
min_samples_list = [3, 5, 8, 10, 12, 15]

for eps in epses:
    for min_samples in min_samples_list:
        model = DBSCAN(eps=eps, min_samples=min_samples)
        labels = model.fit_predict(x_scaled)

        # Number of clusters (excluding noise)
        n_clusters = len(set(labels)) - (1 if -1 in labels else 0)

        # Number of noise points
        n_noise = list(labels).count(-1)

        # Silhouette score (only valid if >1 cluster)
        if n_clusters > 1:
            score = silhouette_score(x_scaled, labels)
        else:
```

```
        score = -1  # invalid case

    db_result.append({
        "eps": eps,
        "min_samples": min_samples,
        "clusters": n_clusters,
        "noise_points": n_noise,
        "silhouette": score
    })

# Convert to DataFrame
df_results = pd.DataFrame(db_result)

# Sort results: prioritize high silhouette, more clusters, fewer noise points
df_results = df_results.sort_values(
    by=["clusters", "noise_points", "silhouette",],
    ascending=[False, True, False]
)

print(df_results)

print("A esp of 1.0 and sample size of 12 produces an accurate estimate of the clusters 3, and
the lowest noisepoints of 55")
```

Terminal:

```
%runfile
C:/Users/dylan/OneDrive/Documents/GRAD_SCHOOL/DAAN_682/HOMEWORK_11/homework
11.py --wdir

There are 210 rows, and 8 columns.

======= DATASET INFO ======
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 210 entries, 0 to 209
Data columns (total 7 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   area                 210 non-null    float64
 1   perimeter            210 non-null    float64
 2   compactness          210 non-null    float64
 3   length_kernel        210 non-null    float64
 4   width_kernel         210 non-null    float64
 5   asymmetry_coefficient  210 non-null    float64
 6   groove_length        210 non-null    float64
```

```
dtypes: float64(7)
memory usage: 11.6 KB
None

======= SUMMARY STATS ======
            area   perimeter  ...  asymmetry_coefficient  groove_length
count  210.000000  210.000000  ...             210.000000     210.000000
mean    14.847524   14.559286  ...               3.700201       5.408071
std      2.909699    1.305959  ...               1.503557       0.491480
min     10.590000   12.410000  ...               0.765100       4.519000
25%     12.270000   13.450000  ...               2.561500       5.045000
50%     14.355000   14.320000  ...               3.599000       5.223000
75%     17.305000   15.715000  ...               4.768750       5.877000
max     21.180000   17.250000  ...               8.456000       6.550000

[8 rows x 7 columns]

======= CORRELATION MATRIX ======
                     area  ...  groove_length
area                 1.000000  ...       0.863693
perimeter            0.994341  ...       0.890784
compactness          0.608288  ...       0.226825
length_kernel        0.949985  ...       0.932806
width_kernel         0.970771  ...       0.749131
asymmetry_coefficient -0.229572  ...      -0.011079
groove_length        0.863693  ...       1.000000

[7 rows x 7 columns]

===== MISSING VALUES =====
area                  0
perimeter             0
compactness           0
length_kernel         0
width_kernel          0
asymmetry_coefficient    0
groove_length         0
dtype: int64

===== DUPLICATE ROWS =====
The total number of duplicate rows is: 0
C:\Users\dylan\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks
```

```
than available threads. You can avoid it by setting the environment variable
OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\dylan\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks
than available threads. You can avoid it by setting the environment variable
OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\dylan\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks
than available threads. You can avoid it by setting the environment variable
OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\dylan\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks
than available threads. You can avoid it by setting the environment variable
OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\dylan\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks
than available threads. You can avoid it by setting the environment variable
OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\dylan\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks
than available threads. You can avoid it by setting the environment variable
OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\dylan\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks
than available threads. You can avoid it by setting the environment variable
OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\dylan\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks
than available threads. You can avoid it by setting the environment variable
OMP_NUM_THREADS=1.
  warnings.warn(
Optimal k based on silhouette score: 2
Adjusted Rand Index (comparison with true labels): 0.481
True number of clusters: 3
KMeans clusters: 2
The k mean model is not yielding expected number of clusters
```

```
C:\Users\dylan\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks
than available threads. You can avoid it by setting the environment variable
OMP_NUM_THREADS=1.
  warnings.warn(
Linkage Method Comparison:

WARD | Silhouette: 0.393 | ARI: 0.797
AVERAGE | Silhouette: 0.376 | ARI: 0.686
COMPLETE | Silhouette: 0.350 | ARI: 0.686

Best linkage method (by silhouette and AIR score) is: WARD
     eps  min_samples  clusters  noise_points  silhouette
1   0.70           5         7            78    0.026536
6   0.75           3         6            44    0.016633
0   0.70           3         6            52    0.014836
18  0.85           3         5            20   -0.021725
7   0.75           5         5            66   -0.022330
2   0.70           8         5           154   -0.268695
12  0.80           3         4            38    0.024672
22  0.85          12         4           120   -0.070188
9   0.75          10         4           153   -0.229995
40  1.00          12         3            55    0.131265
13  0.80           5         3            59    0.154762
20  0.85           8         3            69    0.127868
34  0.95          12         3            69    0.110991
27  0.90          10         3            79    0.175962
41  1.00          15         3            83    0.148776
28  0.90          12         3            90    0.126760
35  0.95          15         3            95    0.110185
21  0.85          10         3            97    0.108128
14  0.80           8         3           100    0.094084
15  0.80          10         3           118   -0.000811
8   0.75           8         3           126   -0.038669
29  0.90          15         3           131   -0.080051
16  0.80          12         3           166   -0.260806
24  0.90           3         2            18    0.012749
31  0.95           5         2            20   -0.034559
52  1.10          12         2            23    0.327038
45  1.05          10         2            29    0.300907
46  1.05          12         2            35    0.280921
53  1.10          15         2            38    0.270430
38  1.00           8         2            38    0.268739
39  1.00          10         2            43    0.254068
```

32	0.95	8	2	47	0.243593
19	0.85	5	2	47	0.237079
33	0.95	10	2	52	0.240107
47	1.05	15	2	57	0.221635
26	0.90	8	2	60	0.217094
23	0.85	15	2	179	-0.181298
10	0.75	12	2	183	-0.202616
3	0.70	10	2	189	-0.235811
42	1.05	3	1	8	-1.000000
48	1.10	3	1	8	-1.000000
36	1.00	3	1	9	-1.000000
43	1.05	5	1	9	-1.000000
49	1.10	5	1	9	-1.000000
37	1.00	5	1	13	-1.000000
30	0.95	3	1	15	-1.000000
50	1.10	8	1	17	-1.000000
51	1.10	10	1	22	-1.000000
44	1.05	8	1	23	-1.000000
25	0.90	5	1	28	-1.000000
17	0.80	15	1	194	-1.000000
4	0.70	12	0	210	-1.000000
5	0.70	15	0	210	-1.000000
11	0.75	15	0	210	-1.000000

A esp of 1.0 and sample size of 12 produces an accurate estimate of the clusters 3, and the lowest noisepoints of 55